Quickdress Sprint 1

ORDER No. 001 7 NOV 2026

ITEM	STORY	OWNER	STATUS
No. 001	Add a piece from camera or library	Marwan	DONE
No. 002	Store category, colour, levels, season	Omar, Nada	DONE
No. 003	Edit and delete a piece	Aya	DONE
No. 004	Filter by season, warmth, formality, comfort	Malk	DONE
No. 005	Fill in the details on the phone	Marwan	DONE

13 BACKLOG ITEMS, S1-01 TO S1-13: REPORT TABLE 3 AND JIRA

Who took each line of the order

TEAM 2 CSCE 3701 SPRINT 1

- **Omar Hassan**

ClothingItem model and its rules

Core contracts: ItemStore, PhotoStore

Event bus and WardrobeManager

a868b5d, 813f121, paired with Nada

- **Nada Salem**

SwiftData and photo-file storage

Wardrobe grid, category and colour filters

Item detail screen

CI on GitHub Actions

Paired with Omar, pushed from his account

- **Aya Noaman**

Edit an item with its saved answers

Delete with a confirmation

The menu on the item page

9ca70f0, 59d77b4

- **Malk Metwally**

Filter by season

Low, medium or high warmth, formality and comfort

Filter sheet and removable chips

Jira: Implement filtering based on attributes

- **Marwan Abouelfotuh**

Microkernel architecture revision

Camera and library capture, permissions

Cut-out and suggestions on the phone

The claim-ticket design, tests, documents

PR #1, PR #2

No. 001 Add a piece

PICKED UP

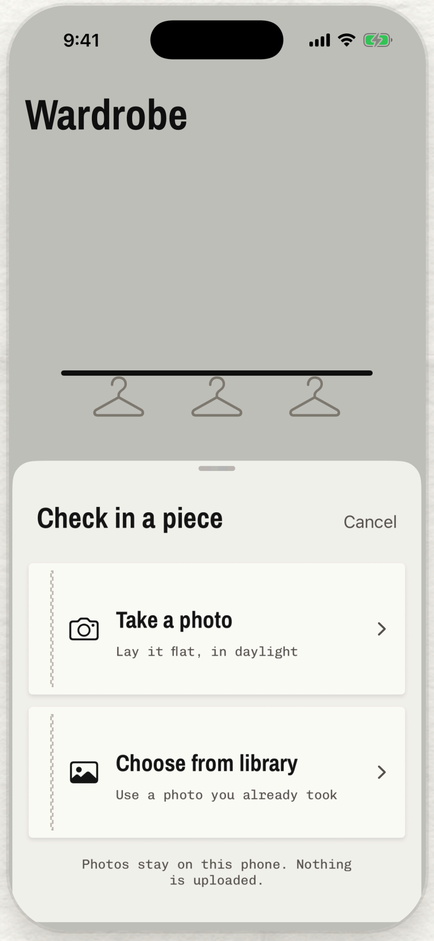
FR-01 NFR-07 MARWAN

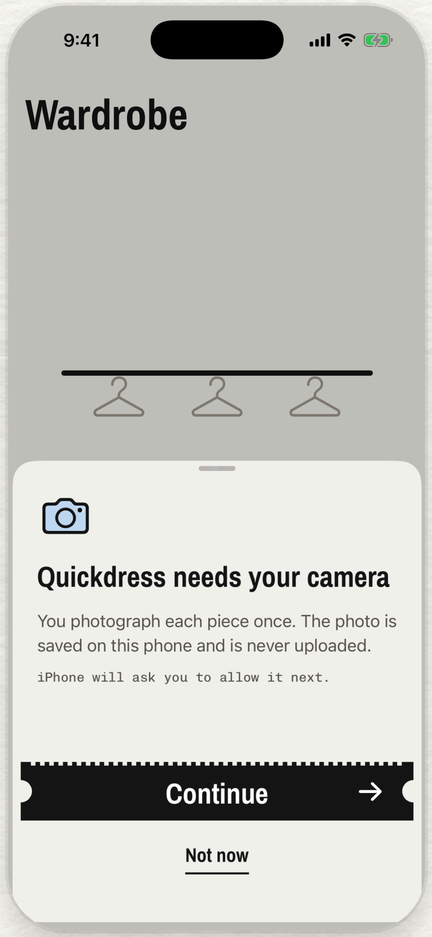
PICKUP SLIP No. 001

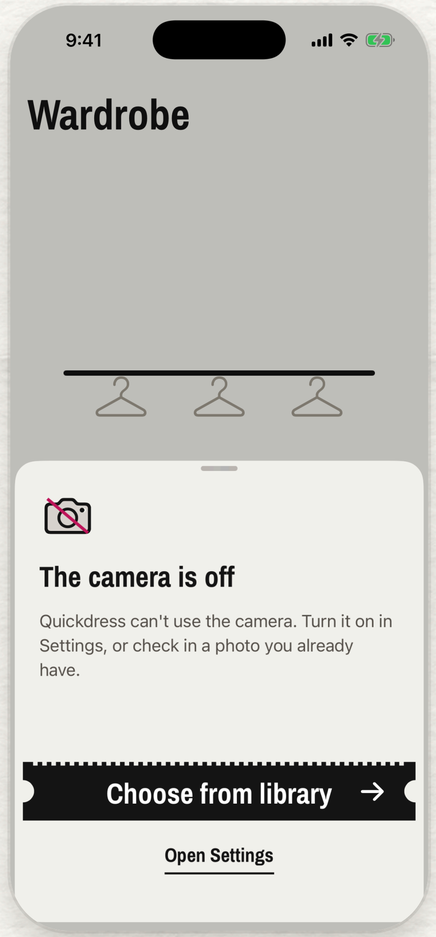
Camera ..... Yes
Photo library ..... Yes
Asks before iOS ..... Yes
Camera refused ..... Still works

The app explains why it needs the camera before iOS asks. If the camera is off, the sheet offers the library and Settings, so nobody gets stuck.

DONE



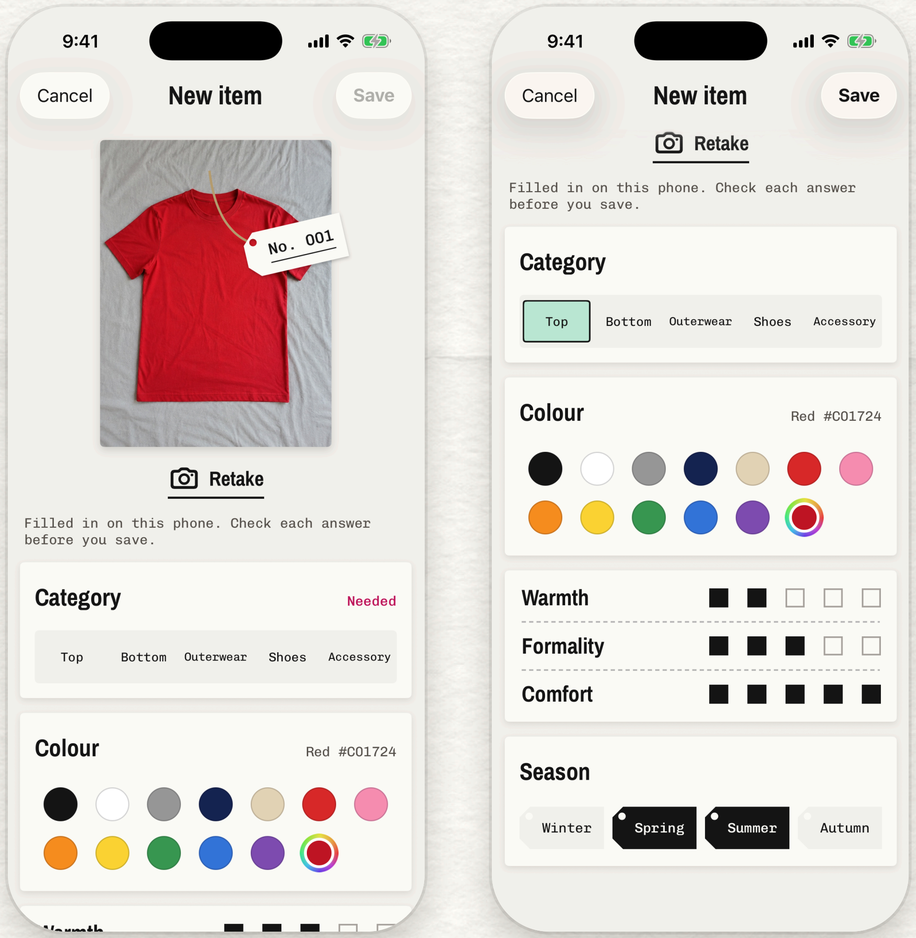




No. 005 Filled in on the phone

PICKED UP

FR-02 FR-13 FR-17 MARWAN



HOW IT WORKS, WITH NO API

Cut-out ····· Apple Vision, on the phone
Category ····· Vision labels, then rules
Colour ····· k-means, nearest in CIE Lab
Exact colour ····· Colour wheel and eyedropper
Ratings, seasons ····· From the category rules

Every answer is marked as a suggestion and can be changed. The screen says so: Filled in on this phone. Check each answer before you save.

The Simulator can't run Vision, so these screens show the full photo and only the colour. On a Mac, the same code cut out all nine test photos (next sheet).

DONE

The cut-out, on nine garment photos

TC-16 1 REAL PHOTO, 8 GENERATED, RUN ON A MAC

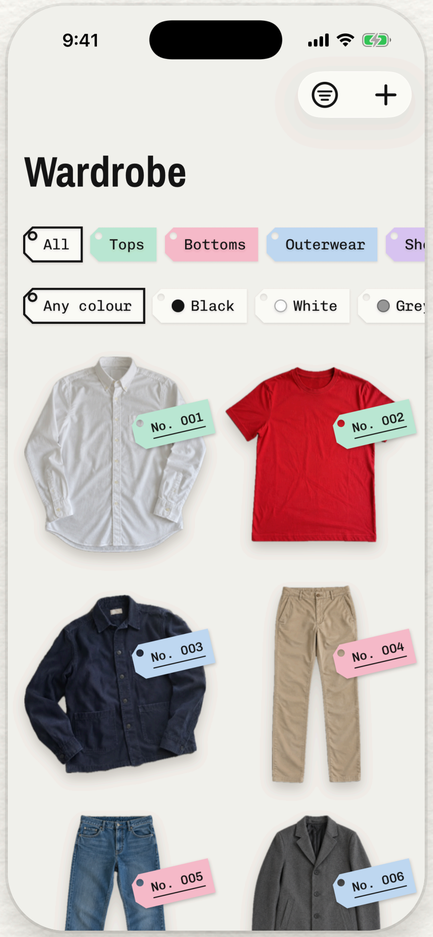
Cut out	9 of 9	Category	6 of 9	Colour, after fix	8 of 9

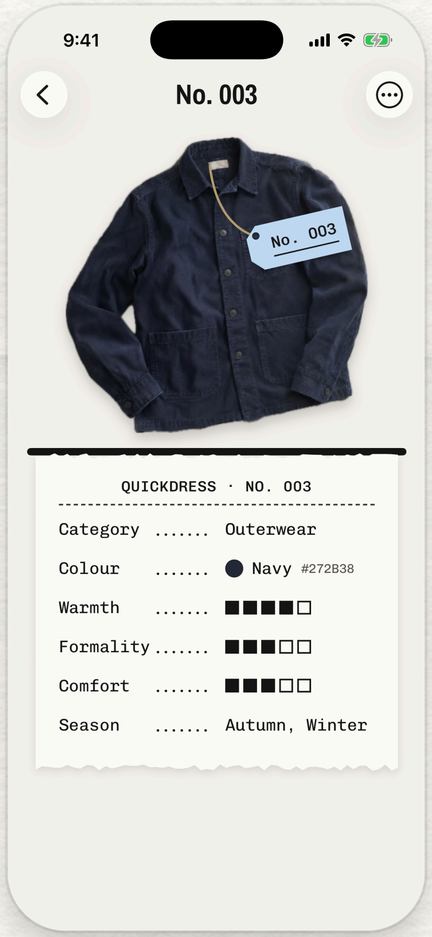
The white sneakers lost part of their white lining with the light background. Light pieces cut out best on a darker surface, and the user can keep the original photo.

No. 002 Store it, browse it

PICKED UP

FR-02 FR-04 OMAR, NADA





WHAT EACH PIECE KEEPS

Category ····· Top, bottom, outerwear, shoes, accessory
Colour ····· 12 palette colours, plus the exact hex
Warmth, formality, comfort ····· 1 to 5
Season ····· At least one
Tag number ····· No. 001 and up

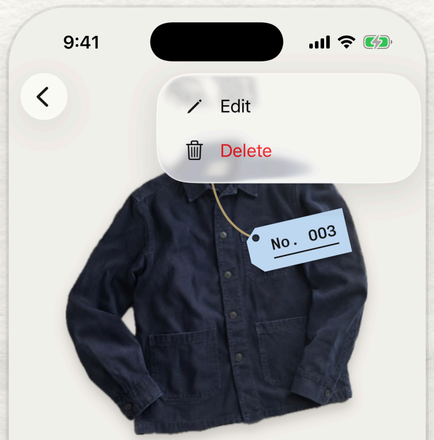
Saved with SwiftData, photos as files on the phone. The grid filters by category and colour with tag-shaped chips; tap a piece for its receipt.

DONE

No. 003 Edit and delete

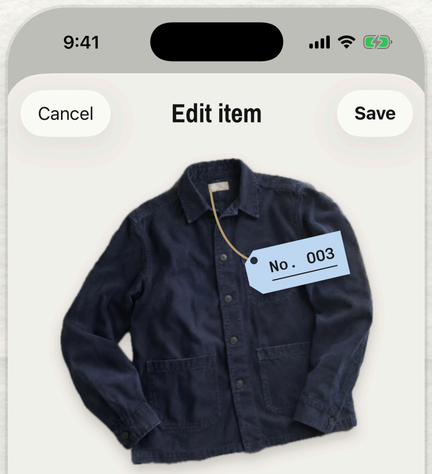
PICKED UP

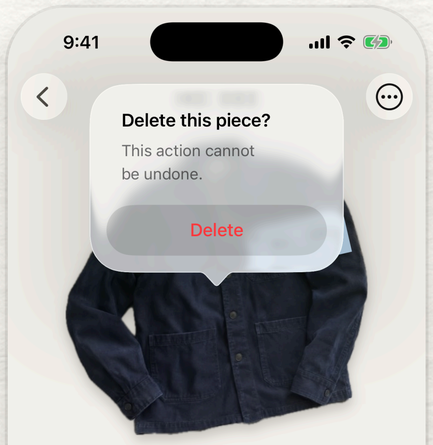
FR-03 AYA



QUICKDRESS · NO. 003

Category ..... Outerwear
Colour ..... Navy #272B38
Warmth ..... ■■■■□
Formality ..... ■■■□□
Comfort ..... ■■■□□
Season ..... Autumn, Winter





QUICKDRESS · NO. 003

Category ..... Outerwear
Colour ..... Navy #272B38
Warmth ..... ■■■■□
Formality ..... ■■■□□
Comfort ..... ■■■□□
Season ..... Autumn, Winter

PICKUP SLIP No. 003

Edit ····· Saved answers
Photo ····· Kept
Delete ····· Asks first
Photo file ····· Removed too

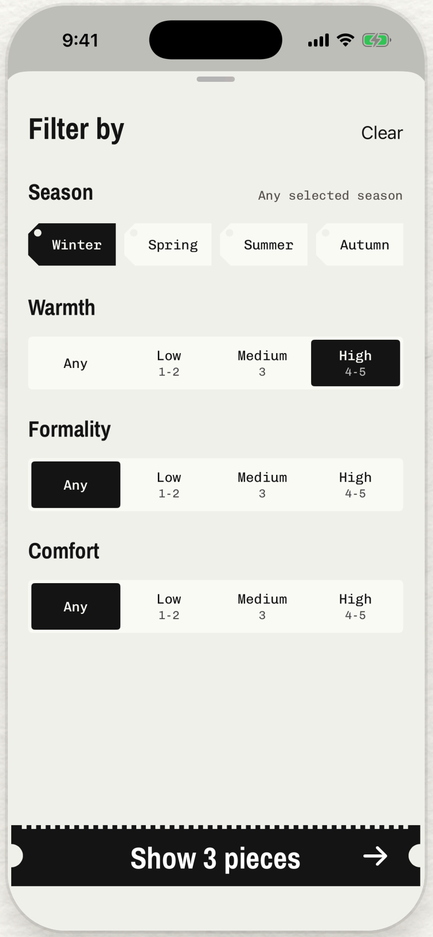
The edit shows on the item page as soon as it is saved. The first push broke the build on main; pull request #2 fixed it and was merged after CI passed.

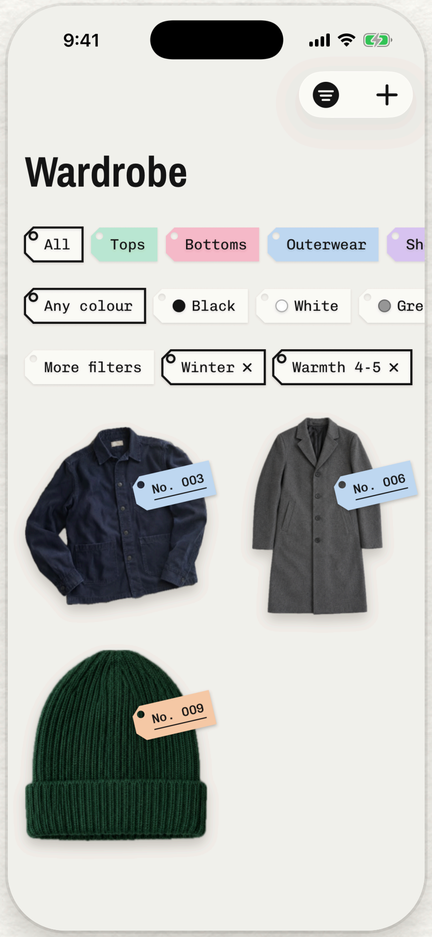
DONE

No. 004 Filter by what it's for

PICKED UP

FR-04 MALK





PICKUP SLIP No. 004

Season ..... Any selected season
Low ..... 1 to 2
Medium 3
High ..... 4 to 5
Combined with ..... Category and colour

In the demo wardrobe of nine photographed garments, winter and high warmth leaves three: the overshirt, the coat and the beanie. The UI test TC-04 uses its own colour-square test wardrobe, where it leaves one.

DONE

The high-level design: a microkernel

ARCHITECTURE DESIGN SPEC 1.1 AND 1.2

Host app SwiftUI screens (MVVM) and the composition root

- **Wardrobe and item screens** S1 — Grid, detail, New item
- **Today screen** — The day's outfit
- **Filters, Settings** — Filters, AI consent
- **QuickdressApp** S1 — Creates and registers every plug-in

Core system, the kernel QuickdressCore, Foundation only

- **WardrobeManager** S1 — Validates and saves items
- **EventBus** S1 — Publish and subscribe
- **PluginRegistry** — Picks plug-ins by priority
- **ContextBuilder** — Weather and calendar
- **OutfitEngine** — Filter pipeline, then recommenders
- **SettingsManager** — Filters, AI consent

- «interface» **ImageSource** S1
- «interface» **ItemStore** S1
- «interface» **PhotoStore** S1
- «interface» **PreferencesStore**
- «interface» **ContextSource**
- «interface» **OutfitFilter**
- «interface» **Recommender**

Plug-ins each realises one contract and never calls another plug-in

- **Capture** S1 — Camera, photo library
- **Storage** S1 — SwiftData, photo files
- **Context** — WeatherKit, Open-Meteo, calendar
- **Filters** — Season, weather, formality, transport
- **Recommenders** — Rule-based; Gemini opt-in

The registry chooses plug-ins at run time by priority and availability, so one can be added or swapped without changing the core or any screen (NFR-08). S1 built in Sprint 1; dashed boxes are planned.

What Sprint 1 built of it

COMPONENT DIAGRAM, SPRINT 1 PART FULL: DESIGN SPEC 7.1

Host app screens call the core only

«component» S1
Wardrobe and item screens
Views and view models (MVVM)

«component» S1
QuickdressApp
Creates every plug-in

«component»
Today screen
Sprint 2

QuickdressCore Foundation only

«component» S1
WardrobeManager
Validates, saves, numbers items

«component» S1
EventBus
itemSaved, itemDeleted

«interface» S1
ImageSource
Camera or library

«interface» S1
ItemStore, PhotoStore
Items and photo files

«component»
PluginRegistry
Sprint 2

Plug-in packages realise the core's contracts

CapturePlugins S1
CameraSource, PhotoLibrarySource
Helpers: SubjectLifter, ItemAnalyzer

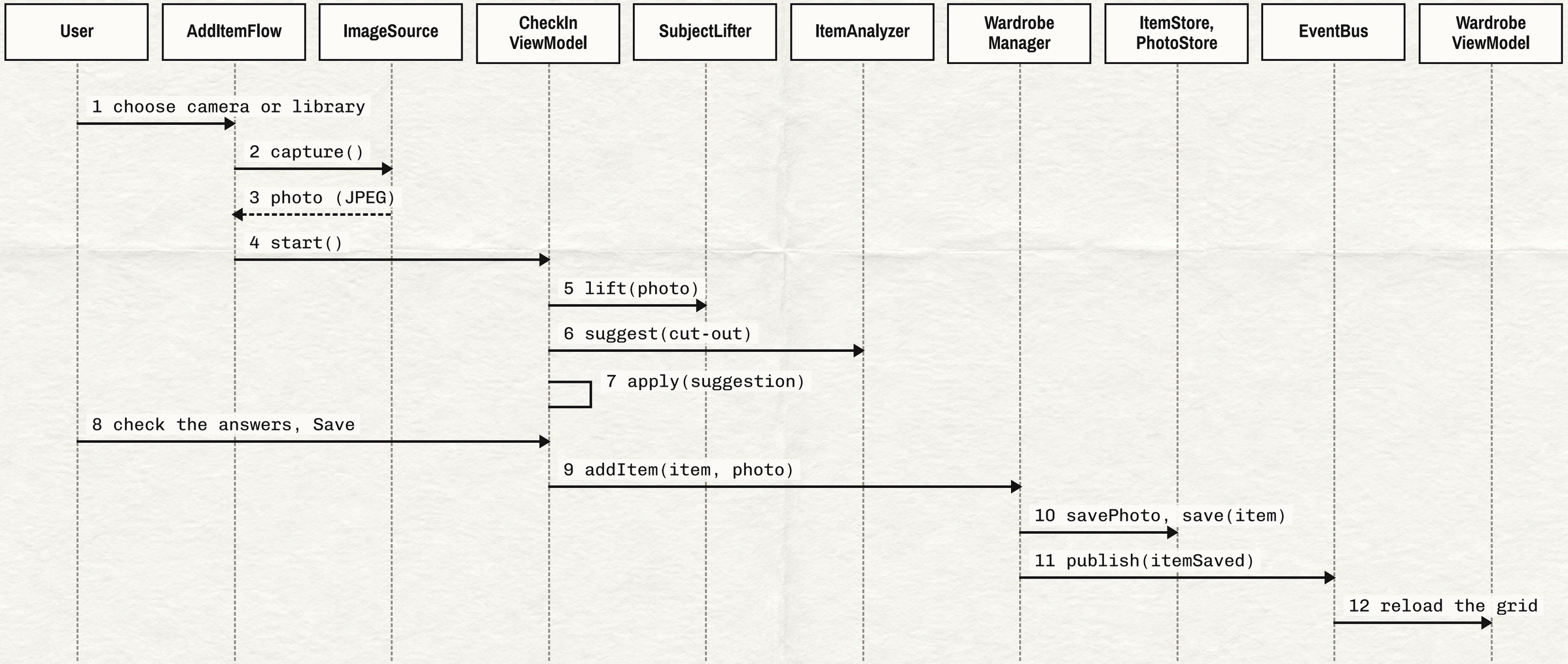
StoragePlugins S1
SwiftDataItemStore, FilePhotoStore
On the phone only

Context, filter, recommender
Weather, rules, outfit
Sprint 2

S1 built in Sprint 1 ⬚ planned The host app uses the core; plug-ins realise its interfaces

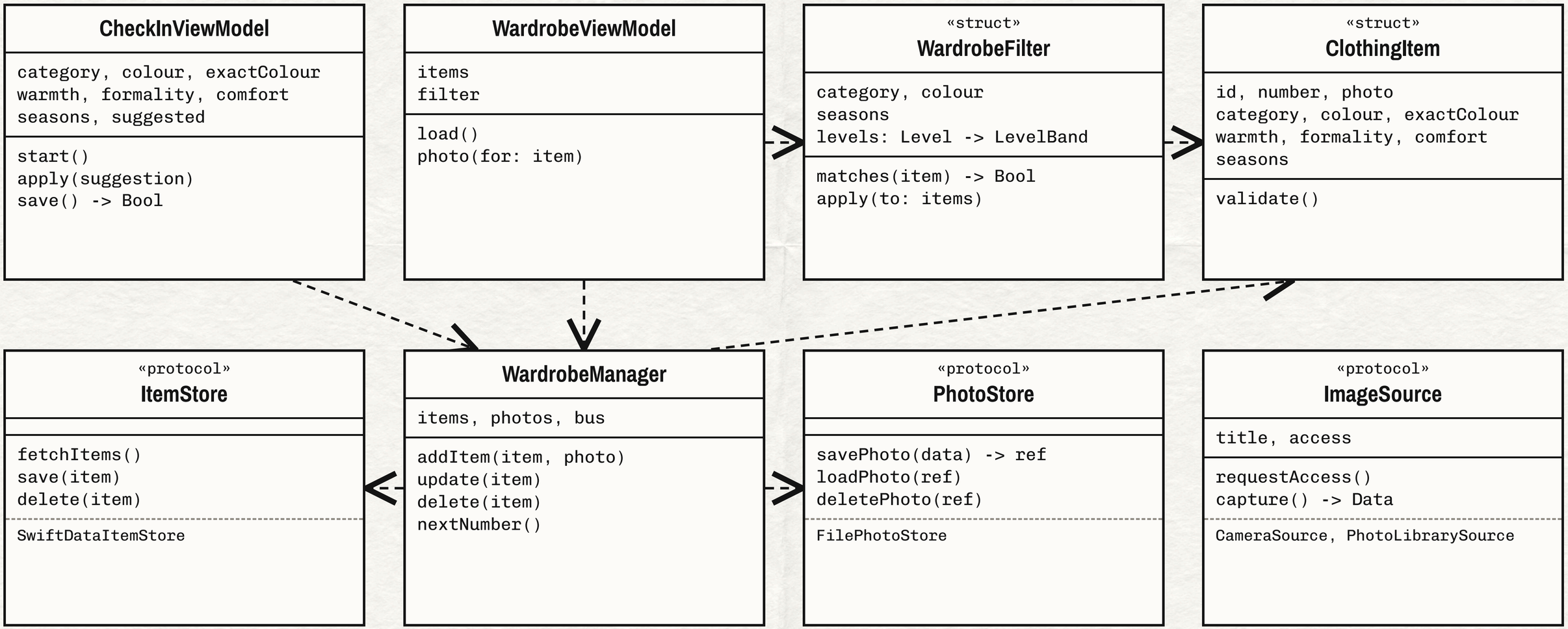
Lower-level design: adding a piece

SEQUENCE DIAGRAM FULL: DESIGN SPEC 7.4



Lower-level design: the code as built

CLASS DIAGRAM, MAIN TYPES FULL: DESIGN SPEC 7.5



Dashed arrows: depends on. Below a protocol: the plug-in that realises it.

How every piece was inspected

TESTING METHOD

LEVEL	WHAT IT CHECKS	TOOL
Unit, white box	Each package alone, with in-memory stores that can be told to fail a save; the screens' view models with no screen	XCTest, swift test
Black box rules	Boundary values on the 1 to 5 ratings (0, 1, 5, 6) and band edges (2 and 3, 3 and 4); equivalence partitions for seasons	XCTest
End to end	The real screens: add, cancel, edit then delete, filter. A test launch mode swaps in an in-memory wardrobe and a test photo source	XCUITest
Manual system	Permission alerts, the real photo picker, relaunch, large text, the cut-out on real photos	Simulator, Mac
Every change	All package tests, the app build and the app's unit tests on every push and pull request	GitHub Actions

The inspection receipt

TEST RESULTS 7 NOV 2026

AUTOMATED TESTS

DONE

QuickdressCore	40
StoragePlugins	14
CapturePlugins	7
App view models	17
End-to-end UI	4
TOTAL	**82 passed**
FAILURES	**0**

MANUAL CASES TC-16 TO TC-22

Passed	3
Failed, fixed, passed	2
Part done (force-quit)	1
Not run (real iPhone)	1

DEFECTS FOUND AND FIXED

D1 main did not build	PR #2
D2 denim read as grey	5 to 8 of 9
D3 tags cut off, large text	capped

FIXED

Notes from the counter

SPRINT RETROSPECTIVE

WENT WELL

- [x] FR-01 to FR-04 work end to end, and FR-13 and FR-17 arrived early
- [x] The core only imports Foundation, so its rules test in milliseconds
- [x] Cut-out and suggestions on the phone, with no API key
- [x] Testing caught three real defects before the demo

TO IMPROVE

- [ ] Build before pushing, and never push to main directly
- [ ] A branch and a reviewed pull request for every change
- [ ] The Simulator has no Vision; we need a signed team iPhone
- [ ] Agree the look before building screens

NEXT SPRINT

- [ ] Branch protection: CI and one review before merging
- [ ] The plug-in registry, with tests
- [ ] FR-05 to FR-08: the daily outfit and the weather
- [ ] Real-iPhone tests, force-quit test, dark mode

The paperwork, updated

REQUIREMENTS AND DESIGN, REVISION 2

REQUIREMENTS SPECIFICATION

FR-02 ····· Tag number, exact colour
FR-04 ····· Season and rating filters
FR-13, FR-17 ····· Delivered in Sprint 1
Sprint plan ····· Sprint 1 done
Use cases ····· Match the built flows

DESIGN SPECIFICATION

Components ····· Built parts marked S1
New section 7 ····· Capture plug-ins, domain
Diagrams ····· Class, add, edit and delete
UI design ····· The claim ticket
Tests ····· As run, with results

Each document opens with a change log of every revision. In the body, yellow rows and paragraphs marked Changed in rev. 2 are the changes, so nothing needs a side-by-side comparison. Revisions 0 and 1 stay in the repository.

Quickdress Sprint 2

ORDER No. 002 OPEN

ITEM	STORY	REQ.	STATUS
No. 006	Plug-in registry, with its tests	NFR-08	OPEN
No. 007	One complete outfit a day, on Today	FR-05	OPEN
No. 008	Swap one piece, or a new outfit	FR-06, FR-07	OPEN
No. 009	The day's forecast sets warmth and coat	FR-08	OPEN
No. 010	Cut-out and suggestions on a real iPhone	FR-13, FR-17	OPEN

Questions?